Audit research notes

Matteo Franzil, Valentino Armani, Luis Augusto Dias Knob, Domenico Siracusa

August 22, 2024

Contents

1 AIs

1.1 DONE SoA research on log clustering, graph representation, and mapping

1.1.1 DONE Inspect SoA for what other people did in the field

- We did some SoA research to understand if other works performed log clustering and pattern matching. The results are quite unsatisfactory: most works - obviously - focus on NLP processing for the log content, few attempt to actually clusterize multiple lines.

  [1] does so in a Hadoop context, a distributed MapReduce system where failures are common. Starting from the assumption that events are limited in quantity, they vectorize sequences over N dimensions (N different events) and clusterize them using similarity.

  [2] LogMiner (first author also wrote [1]) also does something similar to what we want to do: transform log sequences into *behaviours*, depending on the context the various items acted upon. They again use inverse document frequency to assign weight to logs in each cluster, and partitions behaviours using PIDs. The amount of behaviours is, again, quite limited.

1. Q. Lin, H. Zhang, J.-G. Lou, Y. Zhang, and X. Chen, "Log clustering based problem identification for online service systems," in Proceedings of the 38th International Conference on Software Engineering Companion, in ICSE '16. New York, NY, USA: Association for Computing Machinery, May 2016, pp. 102–111. doi: 10.1145/2889160.2889232. Available: `https://dl.acm.org/doi/10.1145/2889160.2889232`. [Accessed: Jun. 18, 2024]
2. H. Zhang, L. Cai, L. Zhao, A. Yu, J. Ma, and D. Meng, "LogMiner: A System Audit Log Reduction Strategy Based on Behavior Pattern Mining," in MILCOM 2022 - 2022 IEEE Military Communications Conference (MILCOM), Rockville, MD, USA: IEEE, Nov. 2022, pp. 292–297. doi: 10.1109/MILCOM55135.2022.10017626. Available: `https://ieeexplore.ieee.org/document/10017626/`. [Accessed: May 09, 2024]

1.2 DONE Implement a rough parser and filter out background noise

1.2.1 DONE Finish clustering non-relevant events and implement them in the parser

We now have a parser that filters out background noise and separates control plane traffic from user traffic. The control plane traffic will be unlabeled, while the user traffic will be labeled depending on the behavior.

1.3 DONE Investigate difference between `kubectl` and other tools

- `kubectl` performs a series of sanity checks before actually sending the request to the API server. This is done by the client itself, for example, getting the namespace and ResourceQuotas before creating an object.
- On the other hand, other behaviours are managed by the `KCM`, for example, creating a secret for a ServiceAccount, or creating a ConfigMap for the root CA.

1.3.1 DONE Try to use curl or an external library

- Does not really change much, user agent apart. Some behaviours are still triggered by the KCM, and API calls made have no effect on his decisions.

1.3.2 FAIL Inspect code to see how api calls are made

- Utter chaos, depending on the command the API calls are different: sometimes it uses the API objects, sometimes raw HTTP requests... plus it uses command builders that are not easy to follow.

1.4 TODO Data generation and collection

1.4.1 DONE Take notes of weird behaviours

Have a look at behaviour analysis for a detailed list of weird behaviours that we have found in the logs.

1.4.2 DONE Collect datasets

- **DONE** Category 1
  - Very basic API objects, e.g. namespaces, pods
- **DONE** Category 2
  - Deployments and similar

- **DONE** Category 3
  - Networking
- **DONE** Category 4
  - Storage, CRDs
- **FAIL** Category 5+
  - All remaining things

1.4.3 TODO Label random dataset

1.5 SCHE Implementation of an ML model for learning behaviours

1.5.1 DONE Label definition

To ease in the labeling and recognition of the data, we have designed a dynamic labeling system that allows us to label data in a more efficient way.

Labels are 22-bit-long integers that encode the following information:

- The type of object that is being acted upon, plus any subtypes
- The action that is being performed on the object
- Whether the object is namespaced or not and if the action is performed on a list of objects or a single object

Labels are not per-line, but rather per-behaviour. For example, all the things that happen when a namespace is created are labeled with the same label: this very label encodes the fact that a namespace (type of object) is being created (action) and that the action is performed on a single object (single object).

To simplify, we have defined a set of "simple" actions that are used to label the data: `create`, `delete`, `get`, `watch`, `update`. These actions combine verbs that are present in the audit logs such as `delete` and `deletecollection` into a single action, allowing us to abstract away from the actual verb used in the log and reduce the label space.

To further reduce the label space, the API specification was used to identify invalid combinations of actions and objects, reducing the number of possible labels to 490.

This label system allows us to easily identify patterns in the data in a quick and efficient way: the parser will be able to inspect the logs and immediately understand what is happening without having to look at the actual log content.

1.5.2 DONE Creation of a pipeline

- The result of our initial work is a pipeline that, taking

raw audit logs as an input, does the following:

- Filters out background noise (e.g. /apis, /healthz, /version)
- Identifies simple control plane behaviours (e.g. service account

token renewal), tags it with `cplabel` to `true` and assigns a label.

- IMPORTANT! This is a stark change to last time. We decided to include all control plane API traffic in the datasets.
- Prompts the user to start the manual labeling process on the filtered data.
- Re-merges the labeled data with the original data.

The data is then ready to be used for training a model.

1.5.3 DONE Model choice

- **FAIL** KNN
  - To avoid diving on difficult and complicated models I first tried something simple (and stupid): K Nearest Neighbors. KNN is a quick and dirty way to classify data, but it was almost instantly clear that it was not going to work.
- **FAIL** CNN
  - I then tried with CNNs, with better yet imperfect results.
    * Accuracy was somewhat OK (90/94%), but the model size quickly spiraled out of control and so were the training times.
    * Convolutional layers usually are interleaved with pooling layers to reduce the size of the data, but in our case the pooling layers made more harm than good. Trying to remove them made the model even bigger, keeping them jeopardized the accuracy.
    * The model was able to pick up on some patterns, but not to generalize well: overfitting was almost guaranteed.
- **DONE** LSTM
  - I then decided to go back to study RNNs and I shortlisted two promising candidates: LSTM and variational autoencoders.
  - So far, LSTM performance has been extremely promising: over 40 randomized attempts, the model has been able to reach an F1 score of 0.98 and accuracy, recall, and precision all over 0.97.
    * P.S. all these metrics have been calculated with the worst possible odds in our favour, e.g., precision and recall with macro averaging.
- **DONE** Bi-LSTM
  - To further improve model capabilities, I switched to a double Bi-LSTM model. The rest of the architecture was left more or less the same, although some Dropout and Batch Regularization layers were added.

1.5.4 SCHE Improve the model

- Work has been done on adapting the model to the new Cat2 data
- Exploratory research on $\text{WINDOW}_{\text{SIZE}}$ (40 -> 60), training splits, and hyperparameters.
- The model also demonstrated to work with completely randomized sequences with Cat1. We need to study this deeper.
- I also worked with changing the output data from an integer class to a tensor.
- Further work is needed with refining features and encodings. A Keras Tuner was used to help with this.
- Final optimization work is focused on reducing as much as possible the errors and finding a suitable metric to replace the F1 score.

1.6 SCHE Implement the final visualizer

- The visualizer takes the output of the model and aggregates actions depending on the label, user, namespace, and so on. The visualizer is able to show the user a timeline of the actions that have been performed in the cluster, and the user can then inspect the actions to understand what is happening.

1.7 Retrieve additional datasets

1.7.1 DELE Ask PoliTO for a better dataset

1.7.2 DONE Ask PerfSPEC Dataset

1.8 Other things

1.8.1 FAIL Scrapped: learn templates from the logs

- Idea: use a ML model to generalize and attempt to match 'templates' of behaviour in the logs. For example: we already have an idea of how namespaces are created, deleted, and so on. We could use a model to learn these templates and then match them in the logs. If no match is found, then an attempt could be made to generate a new template based on the most similar ones.

2 Paper structure

2.1 Story

- Problem: audit logs are hard to set up, use, and understand
- Solution: we propose a system that takes raw audit logs, condenses them into behaviours, and then allows the user to query them in a more user-friendly way.
  - Part 1: logs are pre-processed and filtered
  - Part 2: a ML model assigns labels to the logs
  - Part 3: labels are condensed into behaviours and the user can query or visualize them
- Metrics
  - Space:
    - Metadata logging is not enough, RequestResponse is too much. We try to select the most important parts of the logs and discard the rest [this cannot be done natively], reducing the space needed to store the logs. How many lines are reduced, how much space is saved
    - Other space usage is irrelevant (model is a few MB)
  - Time:
    - Initial training time: done on GPUs takes around ten minutes, depending on the dataset size
    - Inference/filtering time: can be done on commodity CPUs almost in real-time (investigate how)
    - Querying time: to be determined
  - Accuracy
    - model: metrics as already defined
  - Expressiveness
    - how many different behaviours can we detect
    - what queries can be done
  - Usability
    - discussion on hardness to use audit logs vs our system and how much time is saved
- Use cases and metrics on them

2.2 Use cases

After all the metrics have been defined, we can now show some use cases that show how our system can be used in real-world scenarios.

2.2.1 UC1

A system administrator has just recently set up a cluster, but he is unsure if the permissions he set up are correct. He would like to be able to see what is happening in the cluster at a more granular level, for example by filtering per user, per namespace, or per object. This would allow him to see if the permissions he set up are correct, and if not, to be able to correct them.

By using query filters, the administrator can see what is happening in the cluster at a high level, for example by filtering per user, per namespace, or per object. This is not something you can do with regular audit logging: you can indeed filter single lines but you risk missing the context of the action, and also filtering giant logs is not easy as you need to know the structure of the log in advance and write jq queries accordingly.

2.2.2 UC2

A multi-tenant cluster features a large number of users that routinely deploy big Helm charts for testing purposes. The security team has noticed that the number of false alarms generated by the audit system has increased. Indeed, a lot of deployed applications feature namespaces, service accounts, and secrets, which are all legitimate actions but are not easy to differentiate from malicious actions. The security team would like to be able to quickly identify the context of the actions that are being performed in the cluster, and to be able to filter out the background noise generated by the control plane components.

To validate this UC we can test using the top Helm charts, deploying them and showing how to filter and query the logs generated by the deployment.

2.2.3 UC3

The department of a company that is in charge of security has just received a report that a user has been compromised. Sadly, the hacker is not a script kiddie and knows how tools like Falco work: he knows how to avoid detection and may, for example, abuse 'kubectl exec' with 'printenv' to extract secrets from the cluster. Indeed, this behaviour is not detected by Falco and is not easy to detect with traditional alert systems.

To validate this UC we can show that by pinning down a user all the context generated by his actions can be easily retrieved and analyzed.

2.3 Future work

- Continuous learning with CRDs
  - How can a model extend dynamically the set of labels and weights toaccomodate new
- Enabling anomaly detection on the resulting log
  - Instead of feeding the model with the raw logs, we could feed it with the behaviours and see if it can detect anomalies in the patterns.

2.4 Motivation

2.4.1 Reducing the noise

- Traditional alert systems such as Falco provide no cross-log intelligence, and their rules, while accurate, tend to create a large number of false positives. For example,

  - creating a namespace,
  - applying a large Helm chart / YAML which creates a series of objects,
  - legitimate but unpredictable admin actions

  will all create a lot of background noise that Falco will be unable to differentiate.

- This is further exacerbated by some recent works, one that generated a Falco dataset [A] that contains 231K alerts, of which 2K are attack alerts and 229K are normal alerts. The verbosity of the alerts is apparent when we consider that the 2K attack alerts are generated by only a handful of attack scenarios.

[A] Bagheri, S., Kermabon-Bobinnec, H., Majumdar, S., Jarraya, Y., Wang, L., & Pourzandi, M. (2023). Warping the Defence Timeline: Non-Disruptive Proactive Attack Mitigation for Kubernetes Clusters. ICC 2023 - IEEE International Conference on Communications, 777–782. `https://doi.org/10.1109/ICC45041.2023.10278632`

2.4.2 Providing different layers of visibility

- Audit logs are a powerful tool that can be used to understand what is happening in the cluster. However, they are not easy to understand and use. First of all, the configuration of the audit logs, while easy to set up (it's a YAML file and a few flags), requires the administrator to know in advance what he wants to log and the granularity of the logs. This is not always possible, since as we demonstrate later in the paper if you take out some objects from the logs you risk losing the context of the action.

- Second, the logs are HUGE and verbose. A lot of data is generated every second [insert relevant data here], and storage space while cheap is not infinite and can be better used for other purposes.

- Third, the audit logs are hard to filter and understand. They are big YAMLs that contain a lot of information, and filtering them is not easy unless someone knows the structure of the log by heart and is proficient in jq.

3 Quirks and design decisions

3.1 Model details

- Justify the CRDs being put in the 666 sink.

3.2 Control plane components

- Control plane components' "background noise" has been filtered out and the details are available in queries.org.

3.3 Generic

- `watch` requests are long-running and are usually made by control plane components. Since the `ResponseStarted` and `ResponseComplete` events can be in different times, we chose to **remove** the `ResponseStarted` event from the logs.

  - Indeed, for us it is important to have the `requestReceivedTimestamp` that is present in all events anyway. With that, we can understand when the request was made and when it was completed.
  - `ResponseStarted` appears in `watch` requests, in `kubectl exec` requests, and in `delete` requests. In the latter case, `ResponseComplete` is sent once the object is actually deleted.

- `kubectl` requests are internally translated into a set of API calls that are made by the `kube-apiserver`. Further subsections detail such calls and how they can help us derive templates for the parser.

3.4 Namespaces

- Every time a namespace is created, the following things happen:
  - A `list` request is made to `/api/v1/namespaces/{}/resourcequotas` by the `kube-apiserver` for no apparent reason
    - To add insult to injury, usually this line is out of order in the log
  - The `kube-controller-manager` gets the `service-account-controller` SA and generates a token for it
  - The SA `root-ca-publisher` puts the `root-ca.crt` in a ConfigMap in the namespace
  - The `service-account-controller` creates a SA called `default` in the namespace
- When a namespace is deleted, all subresources are deleted with a `deletecollection` request followed by a `list`. Exceptions include `pods`, that weirdly are deleted by a `list`, then `deletecollection`, then `list` again. Also, when `configmaps` are deleted, the `kube-controller-manager` (and not the `root-ca-cert-publisher`) tries to create again the CM that was just deleted, but is rejected because the namespace is being terminated.

3.5 Impersonation

- Users with access to the `impersonate` verb and one of `users`, `groups`, and `serviceaccounts` can impersonate other identities.
- This is correctly reflected in the audit logs, but could be a problem implementation-wise.

3.6 Other quirks

- When a secret is created for a SA the secret is instantly patched by the kube-controller-manager, which is in charge of actually generating that secret along with an object like

  ```
  {"data":{"ca.crt":"...","namespace":"...","token":"..."}
  ```

  where the three fields are base64 encoded.
- Secrets can be traced back to the Pod they have been mounted into by either looking at the spec or monitoring for `watch` requests by the node that is hosting the pod.
- Users with access to `kubectl exec` can access the secrets of the pods they are exec-ing into using `printenv`.
- ResourceQuotas can be used for more than just limiting resources, they can also be used to limit the number of objects that can be created. This also applies to things like `serviceaccounts`: thus, it seems that every time a potentially limitable object is created, a `get` request is made to `/api/v1/{object}` to assess the current state.
- When CSRs are being approved or denied, the `update` verb is used and the actual decision is in the payload of the request. This makes it less straightforward to understand if the request was allowed or denied.
- When old-style ServiceAccount tokens are used, instead of the new CSR style Users, the API server detects the *offending* action and adds a label `kubernetes.io/legacy-token-last-used` to the Secret.